MIT SLOAN SCHOOL OF MANAGEMENT
MIT SCHWARZMAN COLLEGE OF COMPUTING

MODULE 1 UNIT 2
Media Set Video 2 Transcript

© 2026 MIT Sloan & MIT Schwarzman College of Computing
All Rights Reserved

DELIVERED IN COLLABORATION WITH getsmarter™

Module 1 Unit 2 Media Set Video 2 Transcript

Building a customer service assistant

JACOB ANDREAS: Let's start by talking about observations and actions. Here and in the rest of this lesson, we're going to use as a running example the problem of building a customer service assistant that can help users on a website, with returning orders, answering questions, and so on. So how should our interaction begin?

Maybe the user first says, "My package contained the wrong item." They want to process an exchange. The assistant responds by saying, "I'm sorry to hear that. Can you help me by sharing the order number?" The user provides the order number. The assistant can look up the order, but the package was delivered a long time ago and outside the return window. And maybe this is a sort of complicated scenario where, you know, the user had a wedding gift, and they didn't start opening things until after the wedding, and they want the agent to make an exception. The agent has to then consult the sort of company knowledge base and figure out what kinds of exceptions are allowed. And eventually, it figures out that this is permitted, and offers to print a return label and send it to the user.

And the main thing that I want to note here is that to implement this conversation, or even really just to take the first turn in this conversation, our agent needs access to a lot of information that's not in the user's original chat message. For example, they need to be able to look up what other information is associated with the user's order. They need to access information about the company, like what their return policy is or what other information they might need from the user in order to start processing the return, and so on and so forth.

Now, if we go to a sort of standard language model and we ask it to simulate a customer service agent and give it pieces of this conversation, it will generate a plausible-looking conversation. It will even sort of say that it looked up the order and say that it's processing the exchange. But this language model hasn't actually done anything on the background. It's just generated text and is sort of play-acting as an assistant without doing anything out in the world. And so if we want to actually build a customer service agent that can do useful things, we're going to need more than a simple language model.

Adding information

ANDREAS: Now, one other way of making all of this information available to a language model so that it can start to act would maybe be to give it as input, as part of its prompt, absolutely everything that we know. Now, this is going to include not just everything that we know about the current user, but potentially the entire company manual, every customer profile, and every order because we're going to need all of this in order to look up the user and look up their current order and ask them the right kinds of questions.

Now, obviously, we don't actually want to put all of this information into the language model's prompt. Obviously, this is a sort of security nightmare now this language model, and therefore this user, is going to have access to everything that our company knows. And it's going to be a major computational issue because it will be very expensive to keep providing all of this input to the language model at every turn of this conversation and pass it through the neural network over and over again.

© 2026 MIT Sloan & MIT Schwarzman College of Computing
All Rights Reserved

Adding search tools

ANDREAS: So what can we do instead? Well, one thing that we can do is give the language model the ability to search. And so we're going to change now the initial instructions that we provide to it to look something like this. Rather than just saying, "You are a friendly customer service assistant," we will also say, "You can search the help system by writing search_help and typing a query. You can look up pending orders by writing search_orders and typing in an order ID," and so on.

And now, when the user says, "My order contained the wrong item," the language model is going to produce as output, not a response in this conversation, but instead some text that calls this search order function that we put in the input. And what this is going to do is not print a message to the human user, but instead call some sort of external tool that will retrieve the relevant order and place it into the conversation where the language model can see it, along with all of the details of the retrieved order.

And in this example, maybe once it's done that, the language model also calls the search_help function to look up the return policy and finds out that the return window is 30 days. And only after all of this will it call this send_message operation and say to the user, "Okay, I found your order. Unfortunately, it looks like your package was delivered outside the 30-day return window," and so on and so forth.

From text to programs

ANDREAS: Now, the really important thing to understand here is that this language model, at the end of the day, is still just generating text. But by changing the initial instructions that we gave it, it's now generating not text that specifies turns in the conversation, but instead text that specifies small computer programs that call these more complicated programs that can search orders, search help, or send messages optionally to the user.

And so really what we've done is we've changed the way we've interpreted the output from this language model so that it's not just text, but instead arbitrary instructions for interacting with the outside world. And sometimes when it tells us to call a function that will, you know, invoke some sort of search engine, we'll take the results of calling that function and place them back in the conversation for the agent to use at the next turn. And so this is our first example of a language model using a tool.

And you know, like we said, the language model was generating text just as it did before, but now this text has special meaning that causes actions to be taken in the world where that special meaning is provided by whatever scaffolding we wrap this language model in to interpret those actions before presenting them to the users. And so these actions can do things like run searches, take results, and place them in the chat in a way that allows the language model to know what to do next without us needing to provide all of the orders and the entire company base as part of its input all the time.

So far, all of the tool calls that we've seen are for reading data. And so far, our agent hasn't actually done anything that causes changes in the outside world. But once we have the ability to interpret the language model's output as tool calls, we can cause those tools to produce side effects, giving the agent the ability to take actions in the real world as long as those tools, when invoked, cause those actions to occur.

© 2026 MIT Sloan & MIT Schwarzman College of Computing
All Rights Reserved

So, for example, if we create yet another tool and tell the language model about it where this tool now prints a return label, when the language model calls the create_return label tool, we can actually send an instruction to a printer and an automated mailing system and so on, so that the return label is eventually delivered to the current user.

And so what we've built so far is the sort of simplest possible example we can imagine of a language model agent. And what we've done is we've taken a language model, we've let it know that there's a set of programs available for it to run, and it's now going to invoke these programs to take actions in the world to see the effects of these programs and use this to decide what to do next.

SPEAKER: Did you understand all the concepts covered in this video? If you'd like to review any of the content, please click on the chapter menu.

© 2026 MIT Sloan & MIT Schwarzman College of Computing
All Rights Reserved